

### 1. Student Record Management System

Develop a Tkinter-based Student Record Management System using MongoDB to manage student records. Database Name: student

Collection Name: student\_master

Fields: Student\_ID, Name, Course, Semester, Marks of 5 Subjects, Total Marks, Percentage, Grade Requirements:

- Calculate Total Marks, Percentage and Grade automatically.
- Search student by Student ID/Roll No.
- Perform Add, Update, Delete, Search, Display, Clear and Exit operations. e.delete(0,tk.END)

```
from tkinter import *
```

```
from tkinter import messagebox from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/student-last") db = client["school"]
```

```
collection = db["student"]
```

```
def show():
```

```
document = {"name": sn.get(), "std_id": si.get(), "course": cr.get(), "sub5": sm.get()} if collection.find_one({"std_id": si.get()}):
```

```
messagebox.showinfo("Error", "Already exists")
```

```
else:
```

```
collection.insert_one(document)
```

```
messagebox.showinfo("Success", "Inserted")
```

```
def update_data():
```

```
updated = {"name": sn.get(), "course": cr.get(), "sub5":
```

```
sm.get()} collection.update_one({"std_id": si.get()}, {"$set":
```

```
updated}) messagebox.showinfo("Success", "Updated")
```

```
def delete_data():
```

```
collection.delete_one({"std_id":
```

```
si.get()})
```

```
messagebox.showinfo("Success",
```

```
"Deleted")
```

```
root = Tk()
```

root.geometry("300x3")

t = Label(root, text="Register")

t.grid(row=0, column=0, columnspan=2, pady=10)

Label(root, text="Name").grid(row=1, column=0,

pady=10) sn = Entry(root)

sn.grid(row=1, column=1)

Label(root, text="Student\_id").grid(row=2, column=0,

pady=10) si = Entry(root)

si.grid(row=2, column=1)

Label(root, text="course").grid(row=3, column=0, pady=10)

cr = Entryr

Label(root, text="Total 5 sub marks").grid(row=4, column=0,

pady=10) sm = Entry(root)

sm.grid(row=4, column=1)

btn1 = Button(root, text="Submit", command=show)

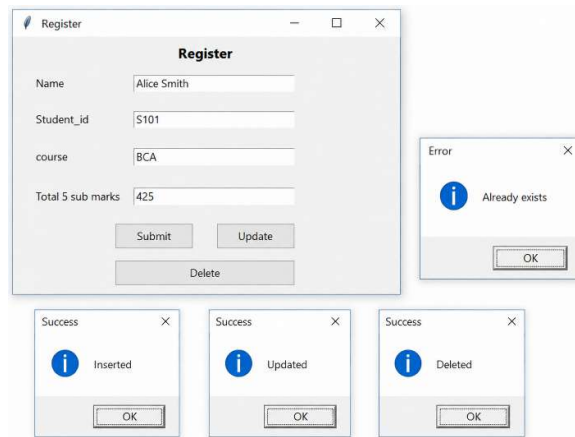
btn1.grid(row=5, column=0, pady=10)

btn2 = Button(root, text="Update", command=update\_data)

btn2.grid(row=5, column=1, pady=10)

btn3 = Button(root, text="Delete", command=delete\_data)

btn3.grid(row=6, column=0, columnspan=2, pady=10)



## 2.Employee Payroll Management System

Develop a Tkinter-based Employee Payroll Management System using MongoDB.

Database Name: employee

Collection Name: emp

Fields: Employee\_Name, Department, Basic\_Salary, HRA\_Percentage, HRA, CHRA, CDA, CPF, Gross\_Salary, Net\_Salary

Calculate the required salary components, Gross Salary and Net Salary automatically. Provide buttons for Insert, Update, Delete, Search, Display, Clear and Exit operations. from tkinter import \*

```
from tkinter import messagebox
from pymongo import MongoClient
```

```
client =
```

```
MongoClient("mongodb://localhost:27017/") db =
```

```
client["payroll_db"]
```

```
collection = db["employees"]
```

```
def calculate_salaries():
```

```
    """Helper logic to safely parse inputs and calculate monetary
    variables.""" try:
```

```
        basic = float(basic_entry.get())
```

```
        or 0) hr_p = float(hr_entry.get())
```

```
or 0) da_p =

float(da_entry.get() or 0) pf_p

= float(pf_entry.get() or 0)

hr_amount = basic * (hr_p /
100) da_amount = basic *
(da_p / 100) pf_amount =
basic * (pf_p / 100)

gross = basic + hr_amount +
da_amount net = gross - pf_amount
return round(gross, 2), round(net, 2)
except ValueError:
    messagebox.showerror("Error", "Please input valid numerical values for salary and percentages.")
    return None, None
def insert_employee():
    if not emp_id_entry.get() or not name_entry.get() or not basic_entry.get():
        messagebox.showerror("Error", "ID, Name, and Basic Salary are required
        fields!") return

    if collection.find_one({"emp_id": emp_id_entry.get()}):
        messagebox.showinfo("Error", "Employee ID already
        exists!") return

    gross, net =
    calculate_salaries() if gross
    is None:
```

```
document = {
    "emp_id": emp_id_entry.get(),
    "name": name_entry.get(),
    "department": dept_entry.get(),
    "basic_salary":
float(basic_entry.get()),
    "hr_percentage": float(hr_entry.get() or 0),
    "da_percentage": float(da_entry.get() or 0),
    "pf_percentage": float(pf_entry.get() or 0),
    "gross_salary": gross,
    "net_salary": net
}
collection.insert_one(document)
messagebox.showinfo("Success", f"Inserted!\nGross: {gross}\nNet: {net}") clear_fields()

def update_employee():
    if not emp_id_entry.get():
messagebox.showerror("Error", "Please enter Employee ID to update calculations!") return
    gross, net = calculate_salaries() if gross is None:
return
    updated_data = {
"name": name_entry.get(),
"department": dept_entry.get(),
"basic_salary": float(basic_entry.get() or 0),
"hr_percentage": float(hr_entry.get() or 0),
```

```
"da_percentage": float(da_entry.get() or 0),  
  
"pf_percentage": float(pf_entry.get() or 0),  
  
"gross_salary": gross,  
  
"net_salary": net  
  
}
```

```
result = collection.update_one({"emp_id": emp_id_entry.get()},  
{"$set": updated_data}) if result.matched_count > 0:  
messagebox.showinfo("Success", f"Updated!\nRecalculated Gross: {gross}\nRecalculated Net: {net}") else:  
messagebox.showerror("Error", "Employee ID not found!")
```

```
def delete_employee():  
if not emp_id_entry.get():  
messagebox.showerror("Error", "Employee ID is mandatory for deletion.") return
```

```
result = collection.delete_one({"emp_id": emp_id_entry.get()})  
if  
result.deleted_count > 0:  
messagebox.showinfo("Success", "Employee Record Deleted!") clear_fields()  
else:  
messagebox.showerror("Error", "Employee ID not found!")
```

```
def clear_fields():  
emp_id_entry.delete(0, END)  
name_entry.delete(0, END)  
dept_entry.delete(0, END)  
basic_entry.delete(0, END)  
hr_entry.delete(0, END)  
da_entry.delete(0, END)  
result = collection.update_one({"emp_id":  
emp_id_entry.get()},
```

```
{"$set": updated_data}}
```

```
if result.matched_count > 0:
```

```
messagebox.showinfo("Success", f"Updated!\nRecalculated Gross:
```

```
{gross}\nRecalculated Net: {net}") else:
```

```
messagebox.showerror("Error", "Employee ID not found!")
```

```
def delete_employee():
```

```
if not emp_id_entry.get():
```

```
messagebox.showerror("Error", "Employee ID is mandatory for deletion.") return
```

```
result = collection.delete_one({"emp_id": emp_id_entry.get()}) if result.deleted_count > 0:
```

```
messagebox.showinfo("Success", "Employee Record Deleted!") clear_fields()
```

```
else:
```

```
messagebox.showerror("Error", "Employee ID not found!")
```

```
def clear_fields(): emp_id_entry.delete(0, END) name_entry.delete(0, END) dept_entry.delete(0, END)
```

```
basic_entry.delete(0, END) hr_entry.delete(0, END) da_entry.delete(0, END) pf_entry.delete(0, END)
```

```
root = Tk()
```

```
root.title("Payroll Management System") root.geometry("380x480")
```

```
title_lbl = Label(root, text="Employee Payroll Entry", font=("Arial", 14, "bold")) title_lbl.grid(row=0, column=0,  
columnspan=2, pady=15)
```

```
fields = [
```

```
("Employee ID:", "emp_id_entry"), ("Full Name:", "name_entry"),
```

```
("Department:", "dept_entry"), ("Basic Salary:", "basic_entry"), ("HR (in %):", "hr_entry"),
```

```
("DA (in %):", "da_entry"),
```

```
("PF (in %):", "pf_entry")
```

```
]
```

```
for idx, (label_text, var_name) in enumerate(fields, start=1):
```

```
Label(root, text=label_text).grid(row=idx, column=0, pady=6, sticky="w", padx=25) globals()[var_name] = Entry(root,  
width=25)
```

```
globals()[var_name].grid(row=idx, column=1, pady=6)
```

```
btn_submit = Button(root, text="Submit", command=insert_employee, width=12)
```

```
btn_submit.grid(row=8, column=0, pady=15, padx=10)
```

```
btn_update = Button(root, text="Update", command=update_employee, width=12) btn_update.grid(row=8,
column=1, pady=15, padx=10)
```

```
btn_delete = Button(root, text="Delete Record", command=delete_employee, width=26) btn_delete.grid(row=9,
column=0, columnspan=2, pady=5)
```

```
root.mainloop()
```

| ID | Name      | Age | D.O.B      | Email              |
|----|-----------|-----|------------|--------------------|
| 1  | Ram       | 25  | 12-10-2020 | ram@gmail.com      |
| 2  | Sam Kumar | 35  | 12-12-2020 | samkumar@gmail.com |
| 3  | Stanley   | 28  | 12-10-2020 | stanley@gmail.com  |
| 4  | Anish     | 25  | 05-01-2020 | anish@gmail.com    |

### 3.Product and Customer Bill Management System

Develop a Tkinter GUI application using MongoDB for maintaining products and calculating the customer bill.

Collection Name: product

Fields: Product\_ID, Product\_Name, Category, Price, Quantity, Discount, GST Requirements:

- Calculate Amount = Price × Quantity.
- Calculate discount amount.
- Calculate GST at 18%.
- Calculate the final customer bill.
- Provide suitable operations to Insert, Update, Delete, Search, Display, Clear and Exit. from tkinter import \*

```
from tkinter import messagebox from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/") db = client["billing_db"]
```

```
collection = db["customer_bills"]
```

```
def calculate_bill():
```

```
    """Helper logic to safely parse inputs and calculate billing totals."""
```

```
    try:
```

```
        qty = int(qty_entry.get() or 0)
```

```
        price = float(amt_entry.get() or 0)
```

```
        discount_p = float(disc_entry.get() or 0)
```

```
        subtotal = qty * price
```

```
        discount_amount = subtotal * (discount_p / 100)
```

```
        final_bill = subtotal - discount_amount
```

```
    return round(final_bill, 2) except ValueError:
```

```
        messagebox.showerror("Error", "Please input a valid whole number for Quantity and numbers for Price/Discount.")
```

```
    return None
```

```
def insert_bill():
```

```
    if not prod_id_entry.get()
```

```
    or not name_entry.get()
```

```
    or not qty_entry.get()
```

```
    or not amt_entry.get():
```

```
        messagebox.showerror("Error", "Product ID, Name, Quantity, and Price Amount are required fields!") return
```

```
    if collection.find_one({"product_id": prod_id_entry.get()}):
```

```
        messagebox.showinfo("Error", "Bill reference for this Product ID already exists!")
```

```
    return
```

```
    final_total = calculate_bill() if final_total is None:
```

```
    return
```

```
    document = {
```

```
        "product_id": prod_id_entry.get(), "product_name": name_entry.get(), "quantity": int(qty_entry.get()),
```

```
        "price_per_item": float(amt_entry.get()),
```

```
        "discount_percentage": float(disc_entry.get() or 0), "total_bill": final_total
```

```
}  
  
collection.insert_one(document)  
  
messagebox.showinfo("Success", f"Bill Generated Successfully!\nTotal Payable: ₹{final_total}") clear_fields()  
  
def update_bill():  
    if not prod_id_entry.get():  
        messagebox.showerror("Error", "Please enter Product ID to update the billing record!") return  
  
    final_total = calculate_bill()  
    if final_total is None: return  
  
    updated_data = {  
        "product_name": name_entry.get(),  
        "quantity": int(qty_entry.get() or 0),  
        "price_per_item": float(amt_entry.get() or 0),  
        "discount_percentage": float(disc_entry.get() or 0),  
        "total_bill": final_total  
    }  
  
    result = collection.update_one({"product_id": prod_id_entry.get()},  
    {"$set": updated_data})  
  
    If  
    result.matched_count > 0:  
        messagebox.showinfo("Success", f"Bill Updated!\nRecalculated Total: ₹{final_total}") else:  
        messagebox.showerror("Error", "Product ID bill record not found!")  
  
def delete_bill():  
    if not prod_id_entry.get():  
        messagebox.showerror("Error", "Product ID is mandatory to delete a transaction.")  
  
    return  
  
    result = collection.delete_one({"product_id": prod_id_entry.get()})  
  
    if result.deleted_count > 0:  
        messagebox.showinfo("Success", "Customer Bill Record Deleted!")
```

```
messagebox.showerror("Error", "Product ID bill record not found!")
```

```
def clear_fields():
```

```
    prod_id_entry.delete(0, END)
```

```
    name_entry.delete(0, END)
```

```
    qty_entry.delete(0, END)
```

```
    amt_entry.delete(0, END)
```

```
    disc_entry.delete(0, END)
```

```
root = Tk()
```

```
root.title("Customer Billing System")
```

```
root.geometry("380x420")
```

```
title_lbl = Label(root, text="Customer Invoice Entry", font=("Arial", 14, "bold"))
```

```
title_lbl.grid(row=0, column=0, columnspan=2, pady=15)
```

```
fields = [
```

```
    ("Product ID:", "prod_id_entry"),
```

```
    ("Product Name:", "name_entry"),
```

```
    ("Quantity:", "qty_entry"),
```

```
    ("Amount (Price/Item):", "amt_entry"),
```

```
    ("Discount (in %):", "disc_entry")
```

```
]
```

```
for idx, (label_text, var_name) in enumerate(fields, start=1):
```

```
    Label(root, text=label_text).grid(row=idx, column=0, pady=8, sticky="w", padx=25) globals()[var_name] = Entry(root, width=25)
```

```
    globals()[var_name].grid(row=idx, column=1, pady=8)
```

```
btn_submit = Button(root, text="Generate Bill", command=insert_bill, width=12) btn_submit.grid(row=6, column=0, pady=15, padx=10)
```

```
btn_update = Button(root, text="Update Bill", command=update_bill, width=12) btn_update.grid(row=6, column=1, pady=15, padx=10)
```

```
btn_delete = Button(root, text="Delete Invoice", command=delete_bill, width=26) btn_delete.grid(row=7, column=0,
columnspan=2, pady=5)
```

```
root.mainloop()
```

#### 4. Library Management System

Create a Library Management System using Python Tkinter and MongoDB. Database Name: BOOK

Collection Name: BOOK\_master

Fields: Book\_ID, Name, Author, Category, Price, Number\_of\_Copies, Issue\_Date Requirements:

- Insert book records.
- Update book records.
- Delete book records.
- Search book by name.
- Display book records.

```
from tkinter import messagebox from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/") db = client["Book_master"]
```

```
collection = db["Books_name"]
```

```
def insert_book():
```

```
if not book_id_entry.get() or not name_entry.get() or not author_entry.get(): messagebox.showerror("Error", "All
fields (Book ID, Name, Author) are required!") return
```

```
if collection.find_one({"book_id": book_id_entry.get()}): messagebox.showinfo("Error", "This Book ID already exists in
the library catalog!") return
```

```
document = {
```

```
"book_id": book_id_entry.get(), "book_name": name_entry.get(), "book_author": author_entry.get()
}

collection.insert_one(document)

messagebox.showinfo("Success", f"Book '{name_entry.get()}' added to catalog successfully!") clear_fields()

def update_book():
    if not book_id_entry.get():
        messagebox.showerror("Error", "Please enter a Book ID to update details!") return

    updated_data = {
        "book_name": name_entry.get(), "book_author": author_entry.get()
    }

    result = collection.update_one({"book_id": book_id_entry.get()},
    {"$set": updated_data}) if result.matched_count > 0:
        messagebox.showinfo("Success", "Book metadata updated successfully!")
    else:
        messagebox.showerror("Error", "Book ID not found in library records!")

def delete_book():
    if not book_id_entry.get():
        messagebox.showerror("Error", "Book ID is required to remove a record.")
    return

    result = collection.delete_one({"book_id": book_id_entry.get()})
    if result.deleted_count > 0:
        messagebox.showinfo("Success", "Book deleted from registry!")
    clear_fields() else:
        messagebox.showerror("Error", "Book ID not found!")

def clear_fields():
    book_id_entry.delete(0, END)
    name_entry.delete(0, END)
    athor_entry.delete(0, END)
```

```
root = Tk()

root.title("Library Management System")

root.geometry("380x320")

title_lbl = Label(root, text="Library Catalog Registry", font=("Arial", 14, "bold"))

title_lbl.grid(row=0, column=0, columnspan=2, pady=15)

fields = [

("Book ID:", "book_id_entry"),

("Book Name:", "name_entry"),

("Book Author:", "author_entry")

]

for idx, (label_text, var_name) in enumerate(fields, start=1):

Label(root, text=label_text, font=("Arial", 10)).grid(row=idx, column=0, pady=8, sticky="w", padx=30)

globals()[var_name] = Entry(root, width=25, font=("Arial", 10))

globals()[var_name].grid(row=idx, column=1, pady=8)

btn_submit = Button(root, text="Add Book", command=insert_book, width=12) btn_submit.grid(row=4, column=0,

pady=20, padx=15)

btn_update = Button(root, text="Update Details", command=update_book, width=12) btn_update.grid(row=4,

column=1, pady=20, padx=15)

btn_delete = Button(root, text="Remove from Catalog", command=delete_book, width=28) btn_delete.grid(row=5,

column=0, columnspan=2, pady=5)

root.mainloop()
```



**5.Student Attendance Management System**

Develop a Student Attendance Management System using Python Tkinter and MongoDB. Database Name: student

Collection Name: student\_master

Fields: Student\_ID, Name, Course, Total\_Lectures, Lectures\_Attended, Attendance\_Percentage Requirements:

- Calculate Attendance Percentage automatically.
- Perform Insert, Update, Delete and Search operations.
- Search student by Student ID or Name.
- Display student attendance records.

```
from tkinter import messagebox from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/")
```

```
db = client["attendance_master"] collection = db["student_attendance"]
```

```
TOTAL_WORKING_DAYS = 200
```

```
def get_percentage(present_days):
```

```
    """Calculates attendance percentage based on total working days."""
```

```
    return round((float(present_days) / TOTAL_WORKING_DAYS) * 100, 2)
```

```
def save_data():
```

```
    if not si.get() or not sn.get() or not sa.get():
```

```
        messagebox.showerror("Error", "All entry fields are required!") return
```

```
    try:
```

```
        pct = get_percentage(sa.get()) document = {
```

```
            "std_id": si.get(),
```

```
            "std_name": sn.get(), "total_attendance": int(sa.get()), "att_percentage": pct
```

```
        }
```

```
        collection.update_one({"std_id": si.get()}, {"$set": document}, upsert=True) messagebox.showinfo("Success",  
f"Saved! Attendance: {pct}%")
```

```
        clear_fields() except ValueError:
```

```
            messagebox.showerror("Error", "Attendance must be a valid number!")
```

```
def search_student(): if not si.get():
```

```
    messagebox.showerror("Error", "Enter Student ID to search!") return
```

```
record = collection.find_one({"std_id": si.get()}) if record:
```

```
    sn.delete(0, END)
```

```
    sn.insert(0, record.get("std_name", "")) sa.delete(0, END)
```

```
    sa.insert(0, str(record.get("total_attendance", "")))
```

```
lbl_pct.config(text=f"Att %: {record.get('att_percentage', 0.0)}%") else:
```

```
    messagebox.showerror("Not Found", "No student found with this ID!")
```

```
def delete_data(): if not si.get():
```

```
    messagebox.showerror("Error", "Enter Student ID to delete!") return
```

```
result = collection.delete_one({"std_id": si.get()}) if result.deleted_count > 0:
```

```
    messagebox.showinfo("Success", "Record Deleted!") clear_fields()
```

```
else:
```

```
    messagebox.showerror("Error", "Record not found!")
```

```
def clear_fields(): si.delete(0, END) sn.delete(0, END) sa.delete(0, END)
```

```
lbl_pct.config(text="Att %: --")
```

```
root = Tk()
```

```
root.title("Attendance Manager") root.geometry("320x300")
```

```
Label(root, text="Std ID:").grid(row=0, column=0, pady=8, padx=10, sticky="w") si = Entry(root, width=15)
```

```
si.grid(row=0, column=1, pady=8)
```

```
btn_search = Button(root, text=" Search", command=search_student) btn_search.grid(row=0, column=2, padx=5)
```

```
Label(root, text="Std Name:").grid(row=1, column=0, pady=8, padx=10, sticky="w")
```

```
sn = Entry(root, width=22)
```

```
sn.grid(row=1, column=1, columnspan=2, pady=8, sticky="w")
```

```
Label(root, text="Total Att:").grid(row=2, column=0, pady=8, padx=10, sticky="w") sa = Entry(root, width=22)
```

```
sa.grid(row=2, column=1, columnspan=2, pady=8, sticky="w")
```

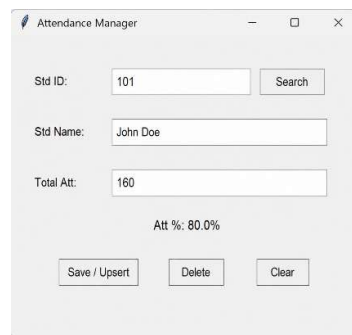
```
lbl_pct = Label(root, text="Att %: --", font=("Arial", 11, "bold"), fg="blue") lbl_pct.grid(row=3, column=0,
columnspan=3, pady=10)
```

```
Button(root, text="Save / Upsert", command=save_data, width=12).grid(row=4, column=0, padx=5, pady=10)
```

```
Button(root, text="Delete", command=delete_data, width=10).grid(row=4, column=1, padx=5, pady=10)
```

```
Button(root, text="Clear", command=clear_fields, width=8).grid(row=4, column=2, padx=5, pady=10)
```

```
root.mainloop()
```



## 6. Bank Account Management System

Develop a simple Bank Account Management System using Python Tkinter and MongoDB. Database Name: Bank

Collection Name: account

Fields: Account\_No, Customer\_Name, Account\_Type (Saving/Current), Deposit\_Amount, Withdraw\_Amount, Balance

Requirements:

- Create/insert a bank account.
- Deposit amount.
- Withdraw amount.
- Calculate and display the current balance.
- Update account details.
- Delete account.
- Search account.
- Display account records.

```
from tkinter import messagebox from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/") db = client["bank"]
```

```
def save_account():
```

```
if not acc_entry.get() or not name_entry.get() or not bal_entry.get(): messagebox.showerror("Error", "All entry fields  
are required!") return
```

```
if collection.find_one({"acc_no": acc_entry.get()}): messagebox.showerror("Error", "Account Number already  
exists!") return
```

```
doc = {
```

```
"acc_no": acc_entry.get(), "customer_name": name_entry.get(), "balance": float(bal_entry.get()), "account_type":  
acc_type.get()
```

```
}
```

```
collection.insert_one(doc)
```

```
messagebox.showinfo("Success", "Account Created Successfully!") clear_fields()
```

```
def search_account():
```

```
account = collection.find_one({"acc_no": acc_entry.get()}) if account:
```

```
name_entry.delete(0, END)
```

```
name_entry.insert(0, account["customer_name"]) bal_entry.delete(0, END)
```

```
bal_entry.insert(0, str(account["balance"])) acc_type.set(account["account_type"])
```

```
else:
```

```
messagebox.showerror("Error", "Account Not Found!")
```

```
def handle_transaction(action_type): try:
```

```
amount = float(trans_entry.get() or 0) if amount <= 0:
```

```
messagebox.showerror("Error", "Enter a valid amount!") return
```

```
account = collection.find_one({"acc_no": acc_entry.get()}) if not account:
```

```
messagebox.showerror("Error", "Search or enter a valid Account Number first!") return
```

```
current_balance = account["balance"]
```

```
if action_type == "deposit":
```

```
new_balance = current_balance + amount elif action_type == "withdraw":
```

```
if amount > current_balance: messagebox.showerror("Error", "Insufficient Balance!") return
```

```
new_balance = current_balance - amount
```

```
collection.update_one({"acc_no": acc_entry.get()}, {"$set": {"balance": new_balance}})
```

```
messagebox.showinfo("Success", f"Transaction successful! New Balance: ₹{new_balance}")
```

```
bal_entry.delete(0, END) bal_entry.insert(0, str(new_balance)) trans_entry.delete(0, END)
```

```
except ValueError:
```

```
messagebox.showerror("Error", "Please enter a valid numeric transaction amount!")
```

```
def clear_fields(): acc_entry.delete(0, END) name_entry.delete(0, END) bal_entry.delete(0, END)
```

```
trans_entry.delete(0, END)
```

```
acc_type.set("Savings")
```

```
root = Tk()
```

```
root.title("Simple Bank System") root.geometry("380x360")
```

```
acc_type = StringVar(value="Savings")
```

```
Label(root, text="Acc No:").grid(row=0, column=0, pady=6, padx=10, sticky="w") acc_entry = Entry(root, width=15)
```

```
acc_entry.grid(row=0, column=1, pady=6, sticky="w")
```

```
Button(root, text="search Find", command=search_account, bg="lightblue").grid(row=0, column=2, padx=5)
```

```
Label(root, text="Customer Name:").grid(row=1, column=0, pady=6, padx=10, sticky="w") name_entry = Entry(root, width=22)
```

```
name_entry.grid(row=1, column=1, columnspan=2, pady=6, sticky="w")
```

```
Label(root, text="Balance:").grid(row=2, column=0, pady=6, padx=10, sticky="w") bal_entry = Entry(root, width=22)
```

```
bal_entry.grid(row=2, column=1, columnspan=2, pady=6, sticky="w")
```

```
Label(root, text="Acc Type:").grid(row=3, column=0, pady=6, padx=10, sticky="w")
```

```
Radiobutton(root, text="Savings", variable=acc_type, value="Savings").grid(row=3, column=1, sticky="w")
```

```
Radiobutton(root, text="Current", variable=acc_type, value="Current").grid(row=3, column=1, padx=80, sticky="w")
```

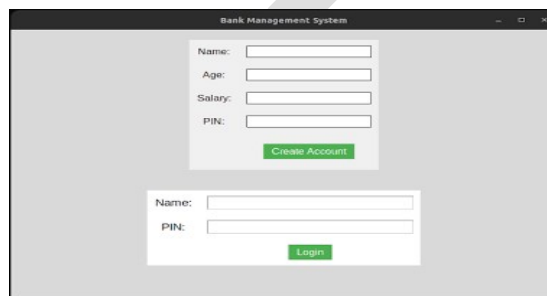
```
Label(root, text="Trans Amount:").grid(row=5, column=0, pady=6, padx=10, sticky="w") trans_entry = Entry(root, width=22)
```

```
trans_entry.grid(row=5, column=1, columnspan=2, pady=6, sticky="w")
```

```
Button(root, text="Create Acc", command=save_account, bg="green", fg="white", width=10).grid(row=6, column=0, pady=10, padx=5)
```

```
Button(root, text="+ Deposit", command=lambda: handle_transaction("deposit"), width=10).grid(row=6, column=1, pady=10, padx=5, sticky="w")
```

```
Button(root, text="- Withdraw", command=lambda: handle_transaction("withdraw"), width=10).grid(row=6, column=1, padx=95, pady=10, sticky="w")
```



## 7.Hospital Management System

Develop a Hospital Management System using Python Tkinter and MongoDB. Database Name: Hospital

Collection Name: patient

Fields: Patient\_ID, Name, Age, Gender, Doctor\_Name, Room\_Type, Number\_of\_Days, Consultation\_Fees, Medicine\_Charges, Room\_Charges, Test\_Charges, Total

Requirements:

- Search patient by Patient ID or Patient Name. from tkinter import \*

```
from tkinter import messagebox from pymongo import MongoClient
```

```
# Database Setup
```

```
client = MongoClient("mongodb://localhost:27017/") db = client["hospital_db"]
```

```
collection = db["patients"]
```

```
def calculate_and_save():
```

```
if not pid.get() or not pname.get():
```

```
messagebox.showerror("Error", "Patient ID and Name are required!") return
```

```
try:
```

```
fees = float(pfees.get() or 0) days = int(pdays.get() or 0) meds = float(pmeds.get() or 0) tests = float(ptests.get() or 0)
```

```
rate = 3000 if rtype.get() == "ICU" else (1500 if rtype.get() == "Private" else 700)
calculated_room_charges = rate * days

grand_total = fees + calculated_room_charges + meds + tests

doc = {
    "patient_id": pid.get(), "patient_name": pname.get(), "room_no": proom.get(), "consultation_fees": fees,
    "room_type": rtype.get(), "number_of_days": days, "medicine_charges": meds,
    "test_charges": tests,
    "room_charges": calculated_room_charges, "total_bill": grand_total
}

collection.update_one({"patient_id": pid.get()},
    {"$set": doc},
    upsert=True)
messagebox.showinfo("Success", f"Record Saved!\nRoom Charges: ₹{calculated_room_charges}\nTotal Bill: ₹{grand_total}")
clear_fields()

except ValueError:
    messagebox.showerror("Error", "Please enter valid numeric values for charges and days!")

def search_patient():
    if not pid.get():
        messagebox.showerror("Error", "Enter Patient ID to search!")
        return

    pat = collection.find_one({"patient_id": pid.get()})
    if pat:
        pname.delete(0, END);
        pname.insert(0, pat.get("patient_name", ""))
        proom.delete(0, END);
        proom.insert(0, pat.get("room_no", ""))
        pfees.delete(0, END);
        pfees.insert(0, str(pat.get("consultation_fees", 0)))
        rtype.set(pat.get("room_type", "General"))
        pdays.delete(0, END);
        pdays.insert(0, str(pat.get("number_of_days", 0)))
        pmeds.delete(0, END);
        pmeds.insert(0, str(pat.get("medicine_charges", 0)))
        ptests.delete(0, END);
```

```
ptests.insert(0, str(pat.get("test_charges", 0)))
```

```
lbl_total.config(text=f"Room Charges: ₹{pat.get('room_charges', 0)} | Total Bill: ₹{pat.get('total_bill', 0)}")
```

```
else:
```

```
messagebox.showerror("Not Found", "No patient record found with this ID!")
```

```
def clear_fields():
```

```
pid.delete(0, END);
```

```
pname.delete(0, END);
```

```
proom.delete(0, END)
```

```
pfees.delete(0, END);
```

```
pdays.delete(0, END);
```

```
pmeds.delete(0, END);
```

```
ptests.delete(0, END) rtype.set("General")
```

```
lbl_total.config(text="Room Charges: ₹0 | Total Bill: ₹0")
```

```
root = Tk()
```

```
root.title("Hospital Billing System") root.geometry("400x440")
```

```
rtype = StringVar(value="General")
```

```
labels = ["Patient ID:", "Patient Name:", "Room No:", "Consultation Fees:", "Number of Days:", "Medicine Charges:",  
"Test Charges:"]
```

```
vars_list = ["pid", "pname", "proom", "pfees", "pdays", "pmeds", "ptests"]
```

```
for i, label_text in enumerate(labels):
```

```
Label(root, text=label_text).grid(row=i if i==0 else i+1, column=0, pady=4, padx=15, sticky="w") globals()[vars_list[i]]  
= Entry(root, width=20)
```

```
globals()[vars_list[i]].grid(row=i if i==0 else i+1, column=1, pady=4, sticky="w")
```

```
Button(root, text="Search", command=search_patient ).grid(row=0, column=1, padx=1)
```

```
Label(root, text="Room Type:").grid(row=3, column=0, pady=4, padx=15, sticky="w") Radiobutton(root,  
text="General", variable=rtype, value="General").grid(row=3, column=1, sticky="w")
```

```
Radiobutton(root, text="Private", variable=rtype, value="Private").grid(row=3, column=1, padx=75, sticky="w")
```

```
Radiobutton(root, text="ICU", variable=rtype, value="ICU").grid(row=3, column=1, padx=145, sticky="w")
lbl_total = Label(root, text="Room Charges: ₹0 | Total Bill: ₹0", font=("Arial", 10, "bold"))
lbl_total.grid(row=9, column=0, columnspan=3, pady=10)
```

```
Button(root, text="Calculate & Save", command=calculate_and_save, width=14).grid(row=10, column=0, padx=10, pady=5)
```

```
Button(root, text="Clear Form", command=clear_fields, width=12).grid(row=10, column=1, pady=5, sticky="w")
```

```
root.mainloop()
```